

บทที่ 3

ภาษาวีเอชดีแอลและขั้นตอนการออกแบบระบบ

3.1 ภาษาวีเอชดีแอล

วีเอชดีแอล (VHDL) ย่อมาจากคำว่า VHSIC Hardware Description Language (VHSIC ย่อมาจาก Very High Speed Integrated Circuit) เป็นภาษาคอมพิวเตอร์ระดับสูง (High Level Language) ภาษาหนึ่งที่ใช้อธิบายการทำงานของระบบฮาร์ดแวร์ สามารถอธิบายฟังก์ชันการทำงานได้หลายๆ ระดับตั้งแต่ระดับบล็อกจนถึงระดับเกต สามารถเขียนความซับซ้อนของระบบได้ตั้งแต่ระดับเกตประกอบกันจนเป็นระบบที่สมบูรณ์ รูปแบบของภาษาวีเอชดีแอลนั้นประกอบด้วย 2 รูปแบบใหญ่ๆ ได้แก่ ภาษาแบบซีควนเชียล (Sequential Language) และภาษาแบบคอนเคอร์เรนต์ (Concurrent Language) การเขียนโปรแกรมด้วยภาษาวีเอชดีแอลมีการเขียนทั้ง 2 รูปแบบรวมกัน เพราะในการทำงานของระบบใดๆ ย่อมจะมีการทั้งทำงานในแบบซีควนเชียลและแบบคอนเคอร์เรนต์ที่อยู่ร่วมกัน นอกจากนี้ตัวภาษาวีเอชดีแอลยังสามารถอธิบายถึงการเชื่อมต่อระหว่างระบบย่อยๆ เข้าไว้ด้วยกันเพื่อให้เป็นระบบใหญ่ได้ ภาษาวีเอชดีแอลนอกจากจะมีการกำหนดรูปแบบไวยากรณ์ของตัวภาษาแล้วยังมีการตรวจสอบความหมายของภาษาว่าจะสามารถจำลองการทำงาน (Simulation) ได้หรือไม่ เพราะโปรแกรมที่เขียนด้วยวีเอชดีแอลต้องมีการตรวจสอบการทำงานด้วย ดังนั้น ในการคอมไพล์จึงมีการตรวจสอบทั้งไวยากรณ์ภาษาและไวยากรณ์การจำลองการทำงาน (Simulation Semantics) อย่างไรก็ตามแม้ตัวภาษาจะมีความซับซ้อนในกฎเกณฑ์ของภาษา แต่การใช้งานก็สามารถทำได้ด้วยการเรียนรู้เพียงบางส่วนของภาษาโดยไม่จำเป็นต้องศึกษารายละเอียดทั้งหมด เนื่องจากตัวภาษาออกแบบมาให้ใช้สำหรับการออกแบบตั้งแต่วงจรที่มีขนาดเล็กไปจนถึงวงจรที่มีขนาดใหญ่และซับซ้อน

3.1.1 ประวัติความเป็นมาของภาษาวีเอชดีแอล

จากการที่มีโครงการวีเอชเอสไอซี (VHSIC) ของกระทรวงกลาโหม (Department of Defense) ของสหรัฐอเมริกาเกิดขึ้น หลายบริษัทที่สร้างชิปวีเอชเอสไอซีได้เข้าร่วมกับโครงการพัฒนาในขณะนั้น ซึ่งแต่ละบริษัทต่างใช้ภาษาเอชดีแอลที่แตกต่างกันไปในการอธิบายชิปของตน ด้วยเหตุนี้ทำให้เกิดความแตกต่างกัน และทำให้แต่ละบริษัทไม่สามารถแลกเปลี่ยนเทคโนโลยีระหว่างกันและกันได้ทำให้กระทรวงกลาโหมเกิดปัญหาในการที่จะพัฒนาและซ่อมบำรุงในภายหลัง จึงต้องการภาษาเอชดีแอลที่เป็นมาตรฐานเพื่อใช้ในการอธิบายการออกแบบต่างๆ โดยได้มอบหมายให้บริษัท 3 บริษัท ได้แก่ IBM Texas Instrument และ Intermetrics ร่วมมือกันพัฒนาและกำหนดมาตรฐานของวีเอชดีแอลขึ้นมาในปี 1983 หลังจากนั้นวีเอชดีแอลเวอร์ชัน 7.2 (VHDL Version 7.2) จึงได้รับการพัฒนาขึ้นและนำออกเผยแพร่สู่สาธารณะในปี 1985 ซึ่งได้รับความสนใจอย่างมากในวงการอุตสาหกรรม โดยเฉพาะอย่างยิ่งบริษัทที่ทำชิปวีเอชเอสไอซี จากผลสำเร็จนี้จึงได้เกิดเป็นมาตรฐาน IEEE ของภาษาวีเอชดีแอลขึ้นในปี 1986 หลังจากนั้นก็มีการพัฒนาขยายขีดความสามารถของภาษาวีเอชดีแอลเพิ่มขึ้น และกระทรวง กลาโหมได้ทำการปรับปรุงและจัดตั้งมาตรฐาน IEEE ของภาษาวีเอชดีแอลใหม่อีกครั้งในปี 1987 เป็นที่รู้จักในชื่อ IEEE.1076

นอกจากนี้ยังทำการกำหนดค่าภายหลังจากเดือนกันยายนปี 1988 บริษัทใดที่ทำการพัฒนา ASIC (Application Specification Integrated Circuit) ให้กับกระทรวงกลาโหมของสหรัฐต้องส่งวีเอชดีโมเดลพร้อมกับชุดทดสอบตามมาตรฐานที่ได้กำหนดไว้

3.1.2 ความสามารถของภาษาวีเอชดีแอล

- สามารถใช้เป็นสื่อกลางในการแลกเปลี่ยนระหว่างผู้ผลิตกับผู้ออกแบบได้
- สามารถใช้เป็นสื่อกลางในการแลกเปลี่ยนระหว่างซีเออี (Computer Aid Electronic) กับซีเอดี (Computer Aid Design) ได้ โดยสามารถคอมไพล์โดยใช้คอมไพเลอร์ และทดสอบด้วยโปรแกรมจำลองการทำงานที่แตกต่างกันได้
- สนับสนุนการออกแบบทั้งแบบบนลงล่าง (Top Down Design) และล่างขึ้นบน (Bottom Up Design) หรือจะออกแบบโดยใช้ทั้งสองแบบร่วมกันก็ได้
- ภาษาวีเอชดีแอลไม่อ้างอิงกับเทคโนโลยีในการออกแบบ ขณะเดียวกันก็ยังสนับสนุนหลายเทคโนโลยี
- สนับสนุนการออกแบบวงจรทั้งแบบเชิงโครนัสและอะซิงโครนัส
- สนับสนุนการออกแบบระบบในหลายๆ วิธีการ เช่น Finite State Machine, Algorithmic, Boolean Equation
- ตัวภาษาสามารถอ่านและทำความเข้าใจได้โดยมนุษย์
- มีมาตรฐานรับรองโดย IEEE และ ANSI ทำให้โมเดลที่ออกแบบมาสามารถเคลื่อนย้ายไปใช้งานยังระบบใดก็ได้ และสามารถนำกลับมาใช้งานได้ใหม่
- สนับสนุนการออกแบบระบบขนาดใหญ่โดยอาศัยความสามารถในการแบ่งส่วนประกอบ (Component), ฟังก์ชัน (Function), โพรซีเจอร์ (Procedure), และแพ็คเกจ (Package)
- สนับสนุนรูปแบบการเขียนถึง 3 รูปแบบ ได้แก่ Behavioral Style, Structural Style และ Data Flow หรือสามารถเขียนทั้ง 3 รูปแบบร่วมกันได้อีกด้วย
- ไม่ต้องศึกษาการเขียนโปรแกรมจำลองการทำงาน เนื่องจากสามารถเขียนโดยใช้ภาษาวีเอชดีแอล
- สามารถเขียนโมเดลให้มีขนาดได้ไม่จำกัดเพราะไม่มีการตั้งข้อจำกัดของตัวภาษาไว้ แต่ทั้งนี้ขึ้นอยู่กับซอฟต์แวร์ที่ใช้เขียน
- สามารถอธิบายตัวแปรที่เกี่ยวข้องกับฟังก์ชันทางด้านเวลา เช่น Propagation Delay, Min-Max Delay, Setup Time, Holding Time, Spike Detection ได้ภายในตัวภาษา
- สามารถสร้างตัวแปรของรูปแบบได้โดยอาศัย Generic
- โมเดลที่ได้จากการออกแบบนั้นไม่เพียงจะอธิบายฟังก์ชันการทำงานเท่านั้น แต่ยังยังสามารถอธิบายถึงรายละเอียด เช่น พื้นที่ทั้งหมด (Total Area) และความเร็ว (Speed) ของตัวโมเดลได้ด้วย
- เป็นมาตรฐานที่ใช้โดยบริษัทผู้ออกแบบหลายแห่งจึงเป็นการทำงานที่จะทำความเข้าใจแม้จะมาจากแหล่งที่แตกต่างกัน

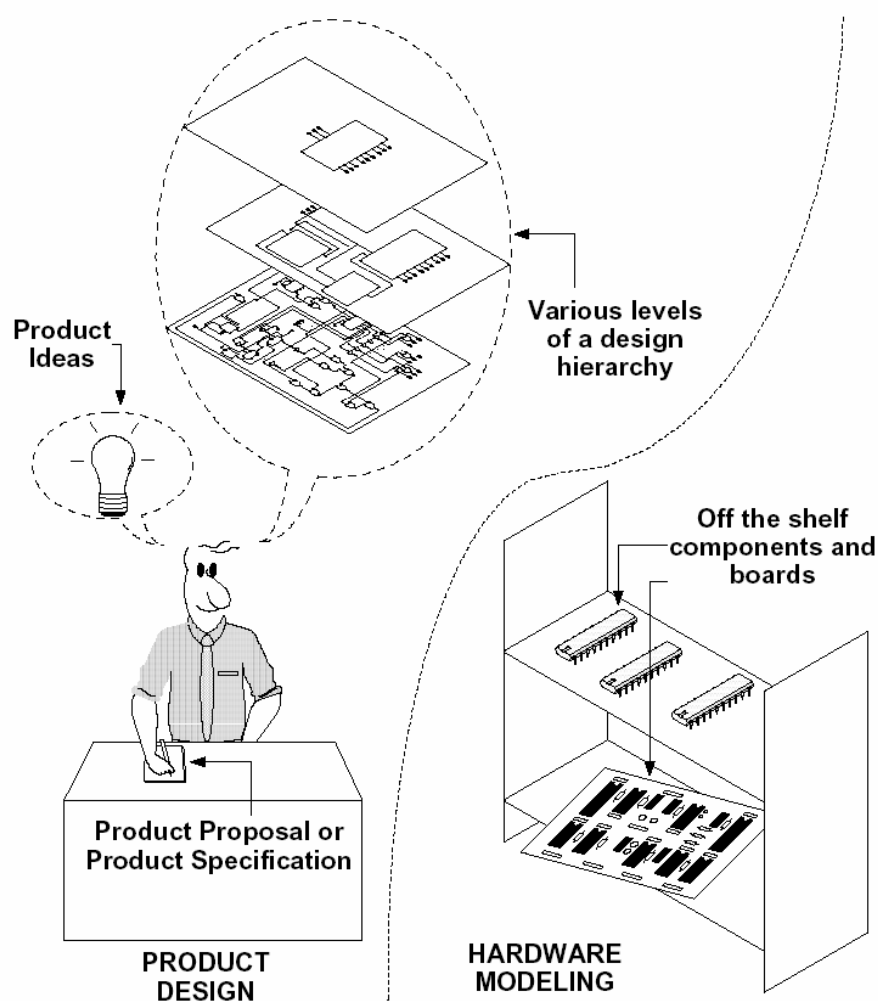
- โมเดลที่สร้างขึ้นสามารถนำไปจำลองการทำงานได้เลย เนื่องจากตัวแปลภาษาได้ทำการตรวจสอบไวยากรณ์ในขोजำลองการทำงานให้ ด้วยแล้ว
- แม้ว่าจะใช้การอธิบายโมเดลแบบ Behavioral ก็สามารถสังเคราะห์ไปเป็นวงจรระดับเกตได้ถ้าทำถูกต้องตามกฎในการสังเคราะห์
- สามารถออกแบบข้อมูลใหม่ๆ ได้ ทำให้เป็นการออกแบบในระดับสูงที่ไม่ต้องคำนึงถึงว่าจะสร้างตัวโมเดลนั้นขึ้นมาอย่างไร

3.1.3 หลักการสร้างโมเดลโดยใช้ภาษาวีเอชดีแอล

วีเอชดีแอล เป็นภาษาที่ใช้สำหรับอธิบายการทำงานของฮาร์ดแวร์ในรูปแบบฟอร์มที่อ่านและทำความเข้าใจง่าย ช่วยในการออกแบบวงจรดิจิทัลและส่วนประกอบต่างๆ โดยอาจใช้เพื่ออธิบายทั้งระบบหรือเพียงบางส่วนในรูปของบล็อก จากนั้นก็จำลองการทำงานโดยรูปแบบดังกล่าวยังไม่ถูกสร้างขึ้นจริงก็ยังคงเป็นเพียงคำอธิบายเท่านั้น หลังจากการจำลองการทำงานจนได้ตามที่ต้องการจึงนำไปทำการสังเคราะห์เพื่อเป็นวงจรระดับเกตต่อไป

ประโยชน์ที่แท้จริงของการใช้ภาษาวีเอชดีแอลในการออกแบบแทนการสร้างต้นแบบขึ้นมา คือการที่เราสามารถอธิบาย Product Idea, Product Proposal, Product Specification ในลักษณะของตัวหนังสือเป็นคำอธิบาย จากนั้นนำไปคอมไพล์เพื่อดูการทำงานของเวลา แล้วทำการแก้ไขจนได้ตามความต้องการเมื่อได้แล้วจึงนำเข้าสู่การสังเคราะห์เป็น Schematic และสร้างเป็นต้นแบบจริงต่อไป ซึ่งต้นแบบจริงที่ได้จะสามารถทำงานได้อย่างถูกต้องแน่นอนเพราะได้มีการทำการจำลองการทำงานมาเป็นที่เรียบร้อยแล้ว ช่วยลดเวลาและค่าใช้จ่ายในการสร้างต้นแบบได้เป็นอย่างมาก

เนื่องจากวีเอชดีแอลเป็นภาษาที่มีประสิทธิภาพสูง การใช้ภาษาวีเอชดีแอลอธิบายฮาร์ดแวร์มีข้อดีที่เด่นชัด 2 ประการ ประการแรกคือ สามารถเข้าใจได้ง่าย เป็นประโยชน์ต่อผู้ที่มีความจำเป็นต้องศึกษาฮาร์ดแวร์ที่ได้มีออกแบบไว้ก่อนแล้วโดยไม่จำเป็นต้องให้ผู้ที่ออกแบบมาอธิบายให้ฟัง เนื่องจากภาษาวีเอชดีแอลนั้นอธิบายการทำงานของมันภายในตัวเองอยู่แล้ว ประการที่สองคือ สามารถแก้ไขได้ง่าย ความสามารถในการแก้ไขได้ง่ายจำเป็นเมื่อต้องการเปลี่ยนแปลงการออกแบบที่ได้ทำไว้แล้ว ซึ่งหากพบว่ามีข้อผิดพลาดที่ต้องแก้ไขหรือเปลี่ยนแปลงความต้องการในระหว่างการออกแบบโดยการเพิ่มเติมการทำงานบางส่วนก็สามารถทำได้ ซึ่งตัวภาษาได้สนับสนุนหลักการในการเขียนแบบต่างๆ ให้สามารถทำการเขียน ปรับปรุงแก้ไขและบำรุงรักษาระบบที่มีความซับซ้อนได้อย่างรวดเร็วและมีประสิทธิภาพ หลักการต่างๆ มีดังต่อไปนี้



รูปที่ 3.1 สิ่งต่างๆ ที่สามารถอธิบายด้วยวีเอชดีแอลได้

3.1.3.1 การออกแบบจากบนลงล่าง (Top Down Design)

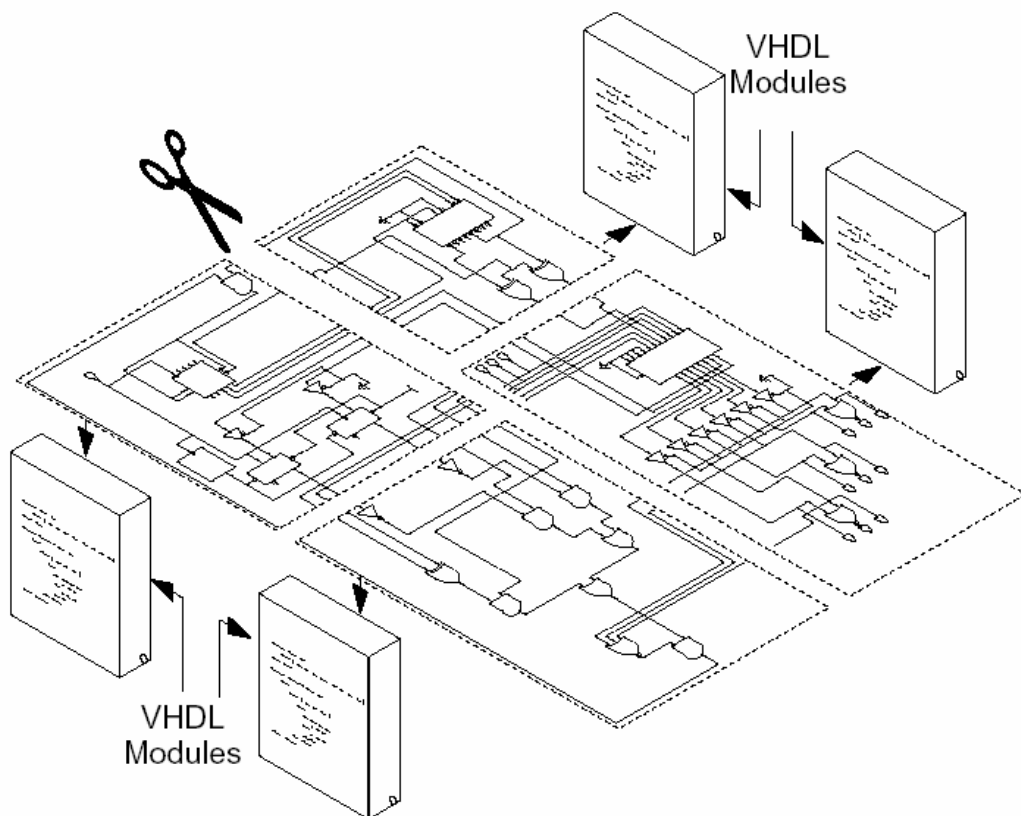
ในการพัฒนางจรดิจิตอลขนาดใหญ่ที่มีความซับซ้อน เช่น ASIC วิศวกรหรือผู้ออกแบบ มักมองรูปแบบระบบให้อยู่ในรูปของ แก้วโครง หรือ บล็อกไดอะแกรม (Block Diagram) เสียก่อนก่อนที่จะลงย่อยในรูปแบบให้ลึกถึงรายละเอียดต่อไป ซึ่งภาษาวีเอชดีแอลได้อนุญาตให้สามารถ

- อธิบายการทำงานของแต่ละบล็อก
- วิเคราะห์การทำงาน (Analyze)
- จัดการแก้ไขและปรับปรุงการทำงานจากผลที่วิเคราะห์ เพื่อให้ได้การทำงานตามที่ต้องการก่อนที่จะทำการออกแบบให้ละเอียดลึกลงในขั้นตอนต่อไป การแก้ไขขั้นตอนนี้จะทำให้ลดค่าใช้จ่ายได้มากกว่าการแก้ไขในช่วงของการพัฒนาเมื่อถึงระดับซิลิกอนชิปแล้ว

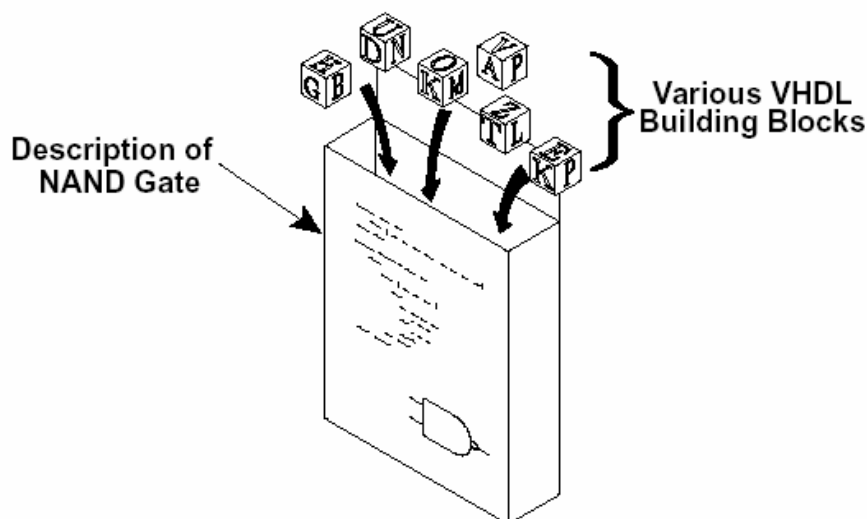
3.1.3.2 การแบ่งเป็นโมดูลย่อย (Modularity)

เป็นหลักการในการแยกส่วน (Partitioning) ฮาร์ดแวร์ออกเป็นส่วนย่อยเล็กลงไป ซึ่งโดยปกติการทำงานของฮาร์ดแวร์ใหญ่จะประกอบด้วยฮาร์ดแวร์ย่อยลงไป ดังรูปที่ 3.2 เป็นรูปแบบซึ่งแสดงวงจรทั้งหมดในรูปเดียว (Flatten Design) หลังจากนั้นค่อยๆ ทำการตัดเป็นส่วนย่อยเล็กลงมา เมื่อเราออกแบบด้วยภาษาวีเอชดีแอล หน้าที่ของการทำงานในแต่ละส่วนสามารถอธิบายได้โดยโมดูลของ ซอร์สโค้ด(คล้ายฟังก์ชันหรือโพรซีเจอร์)ที่การแสดงการทำงานของส่วนย่อยนั้นๆ อย่างชัดเจน การแยกรูปแบบใหญ่ออกเป็นส่วนย่อยนี้ทำให้ง่ายต่อการจัดการและทำความเข้าใจ

ตัวภาษาวีเอชดีแอล ประกอบขึ้นมาด้วยบล็อกโครงสร้างภาษา (Language Building Block) ซึ่งประกอบด้วยคำสงวน (Reserved Word) จำนวน 75 คำ และคำอธิบายโครงสร้าง (Descriptive Word) มากกว่า 200 คำ รูปที่ 3.3 นั้นแสดงให้เห็นว่าภาษาวีเอชดีแอลแต่ละโมดูลนั้นประกอบด้วยบล็อกโครงสร้างภาษาอะไรบ้าง และคำอธิบายการทำงานของเนนด์เกท



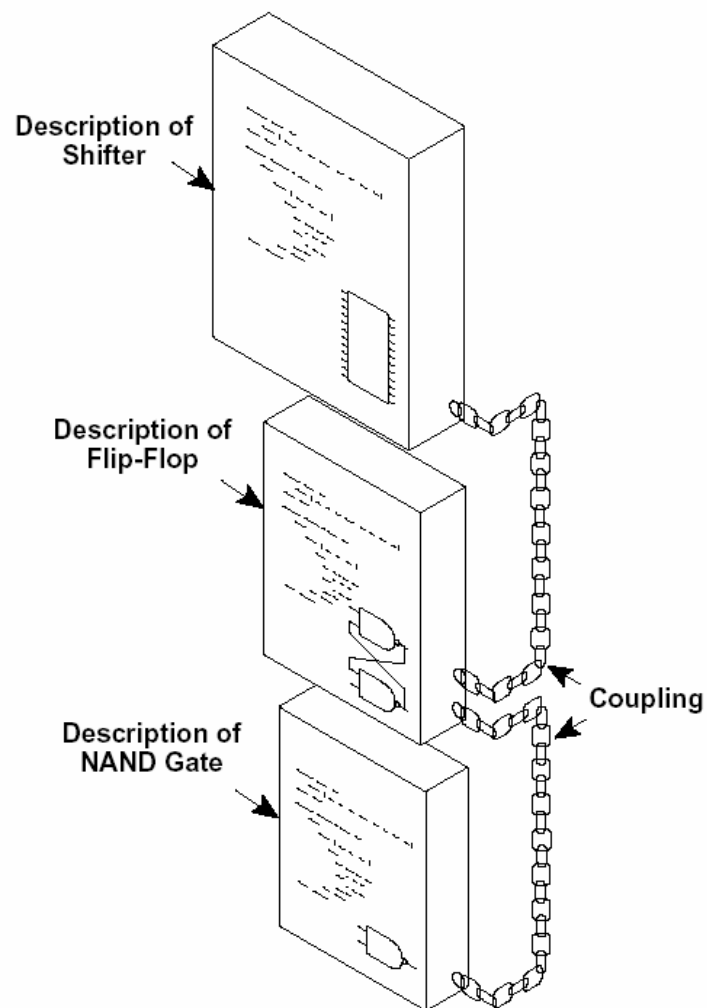
รูปที่ 3.2 การแบ่งย่อยในระดับราบของการออกแบบฮาร์ดแวร์



รูปที่ 3.3 โมดูลของฮาร์ดแวร์ที่ได้จากการสร้างบล็อกโครงสร้างของวีเอชดีแอล

จากรูปที่ 3.4 แสดงวิธีการจัดลำดับชั้น (Hierarchical Method) โดยการแยกส่วนการทำงานออกเป็น ส่วนย่อย ส่วนบนสุดเป็นการอธิบายการทำงานของตัวเลื่อนข้อมูล (Shifter) ส่วนล่างถัดลงมาแสดงการแสดงผล ส่วนของตัวเลื่อนข้อมูลที่อาศัยการทำงานของฟลิปฟล็อป โดยถัดมาฟลิปฟล็อปก็อาศัยการทำงานของ 낸ดเกทอีกทีหนึ่ง ตัวเลื่อนข้อมูลอธิบายการทำงานได้โดยการต่อกันของฟลิปฟล็อป ในระดับต่ำลงมาฟลิปฟล็อปก็เกิดจากการใช้ 낸ดเกทต่อกัน 2 ตัว ในระดับต่ำลงมาอีกเป็น 낸ดเกทซึ่งมีการอธิบายการทำงานของมัน ไว้ภายใน แต่ละโมดูลของตัวเลื่อนข้อมูลจึงมีคำอธิบายการทำงานของตัวเองอยู่แล้ว คำอธิบายมีไว้เพื่อให้สามารถใช้งานโมดูลฟลิปฟล็อปได้ โมดูลฟลิปฟล็อปก็มีการอธิบายการทำงานของตัวเองให้สามารถเรียกใช้งานโมดูล 낸ดเกทในระดับล่างสุดๆได้

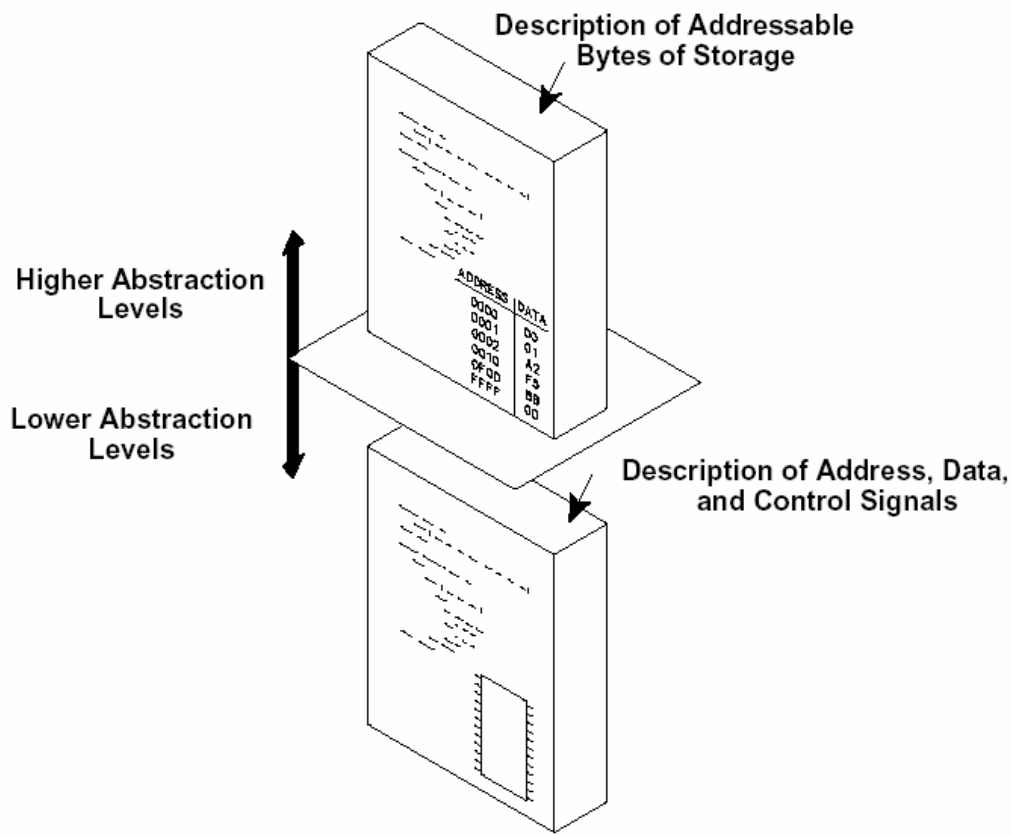
ประโยชน์อย่างหนึ่งของการแยกส่วนฟลิปฟล็อปและ 낸ดเกทออกจากกัน คือการทำให้ง่ายต่อการใช้งาน 낸ดเกทตัวนี้ในบล็อกของโมดูลตัวอื่นๆ ที่อยู่ในระดับสูงกว่า ทำให้นำออกมาใช้ได้ อีกหลายครั้งโดยลดความซับซ้อนในการเขียนอุปกรณ์ลง และเป็นการง่ายที่จะแก้ไขการทำงานของตัวเลื่อนข้อมูลโดยปราศจากการแก้ไขการทำงานของฟลิปฟล็อปและ 낸ดเกท จากประโยชน์ของการแบ่งเป็นโมดูลย่อยนี้ทำให้การออกแบบง่ายต่อความเข้าใจและแก้ไขได้ตามต้องการ



รูปที่ 3.4 การแบ่งลำดับชั้นของการอธิบายตัวเลื่อนข้อมูลโดยใช้วีเอชดีแอล

3.1.3.3 การมองการทำงานแต่ละส่วน (Abstraction)

เป็นการอธิบายการทำงานมากกว่าอธิบายว่าจะพัฒนาอย่างไร หลักการนี้สัมพันธ์ใกล้ชิดกับหลักการทำเป็นโมดูลย่อยในหัวข้อที่ผ่านมา จะเห็นได้ว่าตัวฟลิปฟล็อบนั้นถูกอธิบายในรูปของแนนด์เกต ขณะที่ตัวเลื่อนข้อมูลนั้นถูกอธิบายในรูปของฟลิปฟล็อป แต่ละระดับของการออกแบบนั้นถูกสร้างจากในระดับล่างขึ้นมาทั้งสิ้น

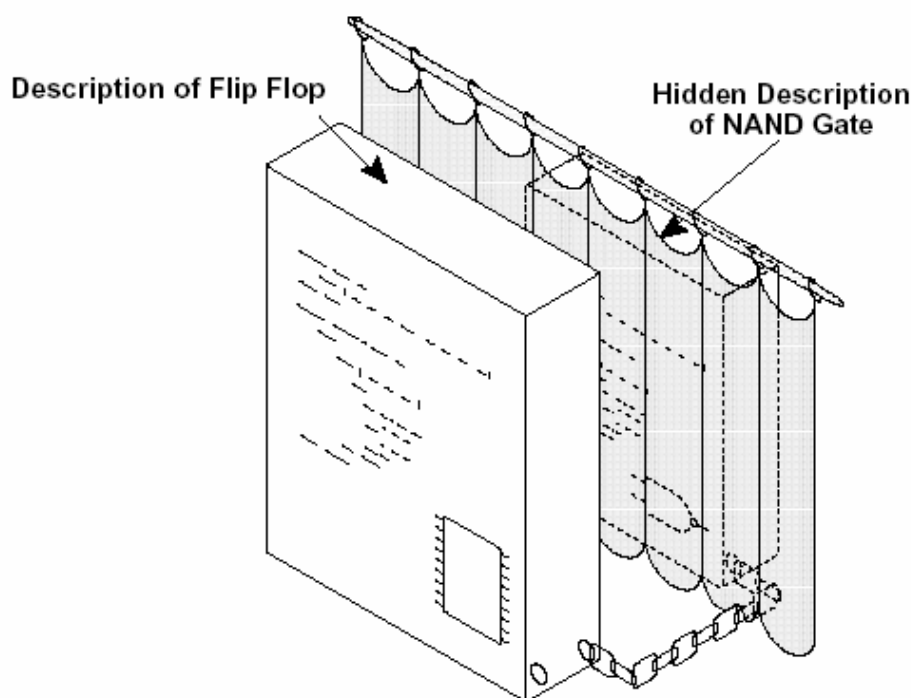


รูปที่ 3.5 การมองการทำงานแต่ละส่วนของการอธิบายหน่วยความจำรวม

รูปที่ 3.5 แสดงให้เห็นถึงอีกวิธีการหนึ่งของการอธิบายการทำงานโดยใช้ภาษาวีเอชดีแอลในระดับของการออกแบบ ในการนิยามหน่วยความจำแบบรวมโดยอาศัยการใช้ภาษาระดับสูง แสดงถึงตำแหน่งสำคัญต่างๆ ซึ่งข้อมูลถูกเก็บไว้ในตำแหน่งนั้นๆ ที่ระดับนี้ไม่ต้องสนใจถึง Address Line, Data Line หรือ Control Line เราสามารถเพ่งจุดสนใจไปที่ขนาดของข้อมูล โดยไม่ต้องสนใจถึงสัญญาณควบคุมต่างๆ ภายใน เพราะส่วนดังกล่าวจะถูกจัดการในระดับที่ต่ำลงมา ในระดับล่างเราสามารถอธิบายการทำงานของสัญญาณแต่ละเส้นภายในรวม การจัดการสัญญาณภายในทุกเส้น การอ่านข้อมูลหรือเขียนข้อมูลในรวม ดังนั้นถ้าต้องการเปลี่ยนค่าของข้อมูลในรวม เราควรแก้ไขในระดับที่สูงขึ้นมาจะทำให้ง่ายกว่าการแก้ไขสัญญาณควบคุมภายใน ซึ่งเราจะเห็นได้ว่าในแต่ละระดับมีความเหมาะสมแตกต่างกันไป **จุดนี้เองที่ให้ออกแบบไว้** **ง่ายต่อการแก้ไข**

3.1.3.4 การซ่อนรายละเอียดที่ไม่จำเป็น (Information Hiding)

เมื่อทำการเขียนฮาร์ดแวร์วีเอชดีแอลขึ้นมา เพื่ออธิบายการทำงานของฮาร์ดแวร์ตัวหนึ่ง บางครั้งเราอาจต้องการซ่อนรายละเอียดของการพัฒนาโมดูลนั้นๆ โดยไม่ให้โมดูลอื่นรู้การทำงานของโมดูลดังกล่าว การซ่อนรายละเอียดของภาษาวีเอชดีแอลทำให้มีการจัดการได้ง่าย และสามารถอ่านและทำความเข้าใจได้ง่ายขึ้น และเป็นการสนับสนุนหลักการที่ผ่านมา คือ การมองว่าแต่ละส่วนมีทำงานอย่างไรใช้งานอย่างไรมากกว่าที่จะสนใจว่ามัน



รูปที่ 3.6 การซ่อนรายละเอียดที่ไม่สำคัญของระดับแนนด์เกต

ถูกสร้างมาขึ้นได้อย่างไร ประกอบด้วยวงจรอะไรบ้าง การซ่อนรายละเอียด ทำให้ความสนใจของผู้ออกแบบมุ่งไปในส่วนที่สำคัญของโมดูลมากกว่าโดยซ่อนส่วนที่ไม่สนใจไว้ไม่ให้มองเห็นและไม่สามารถเข้าถึงได้ จากรูปที่ 3.4 คำอธิบายการทำงานของแนนด์เกตนั้นจะถูกปิดเอาไว้จากผู้ที่ออกแบบ เมื่อถึงผู้ที่ออกแบบพลิกฟลอปก็ไม่ต้องสนใจว่าแนนด์เกตจะทำงานอย่างไร สามารถนำมาใช้งานได้เลย โดยแนนด์เกตที่ออกแบบไว้แล้ว สามารถคอมไพล์แล้วเก็บไว้เป็นไลบรารี สำหรับผู้ที่ออกแบบระดับสูงขึ้นรู้เพียงแค่ว่าจะทำการเชื่อมต่อสัญญาณอินพุตและเอาที่พุทกับแนนด์เกตนั้นมาใช้ได้อย่างไร ประโยชน์ที่สำคัญอีกอย่างหนึ่งคือการป้องกันข้อมูลภายในของการออกแบบ ในกรณีที่มีการแจกจ่ายโมเดลวีเอสดีแอลไปยังที่อื่น เช่น ส่งไปทำการพัฒนาร่วมกับโมเดลอื่น ก็อาจทำได้โดยการส่งโมเดลที่ทำการคอมไพล์แล้วไม่ต้องส่งซอร์สโค้ดไปให้ เป็นการป้องกันทรัพย์สินทางปัญญาได้ในระดับหนึ่ง

3.1.3.5 การมีรูปแบบเป็นแบบแผน (Uniformity)

การมีรูปแบบที่เป็นแบบแผน เป็นหลักการหนึ่งของการอธิบายฮาร์ดแวร์ด้วยภาษาวีเอสดีแอล หมายถึงการสร้างโมดูลของซอร์สโค้ดลักษณะคล้ายกัน โดยรูปอยู่ในรูปแบบบล็อกโครงสร้างภาษาวีเอสดีแอล (VHDL Building Block) ทำให้การเขียนซอร์สโค้ดมีโครงสร้างที่ดี เช่น มีการใช้ย่อหน้า การใช้คำอธิบาย เป็นต้น เป็นผลให้มีการพัฒนาโมดูลที่อ่านและทำความเข้าใจได้ง่าย

3.2 ขั้นตอนการออกแบบระบบด้วยภาษาวีเอชดีแอล

3.2.1 Design Analysis and Specification

เป็นขั้นตอนในการออกแบบ วิเคราะห์และหาความต้องการของระบบ โดยใช้แนวความคิด และหลักการในการแก้ปัญหา แล้วรวบรวมและเขียนเป็นความต้องการของระบบ ตลอดจนวางแผนในการทดสอบความถูกต้องของการออกแบบระบบ ทั้งวิธีการในการทดสอบ และค่าทดสอบ (Test Vector) ที่ต้องใช้ในการทดสอบ

3.2.2 Coding

เป็นการนำเอาความต้องการที่ได้จากขั้นตอนแรกมาทำการเขียน โปรแกรมภาษาวีเอชดีแอลเพื่อบรรยายวงจรในรูปแบบของภาษายบรรยายการทำงานระดับฮาร์ดแวร์ โดยสามารถเขียนได้ในหลายรูปแบบ เช่น แบบบรรยายโครงสร้าง แบบบรรยายพฤติกรรม หรือแบบ RTL หรือใช้ทั้ง 3 แบบร่วมกันก็ได้

3.2.3 Functional Simulation

เป็นการจำลองการทำงานของวงจรที่ได้ออกแบบไว้ด้วยโปรแกรมจำลองการทำงาน (Software Simulator) โดยโปรแกรมจะนำ Test Vector ที่ต้องการทดสอบป้อนให้กับวงจรที่เขียนขึ้นแล้วจำลองเอาผลลัพธ์การทำงานออกมาอยู่ให้เห็นในรูปของ Waveform โดยสามารถทำได้ 2 วิธี คือ

3.2.3.1 กำหนดค่าให้กับโปรแกรมจำลองการทำงานเอง

เป็นการกำหนดสัญญาณให้กับโปรแกรมจำลองการทำงานด้วยการ Force Input Signal โดยจะต้องทำทุกครั้งที่มีการทดสอบ เหมาะกับการทดสอบการทำงานของวงจรขนาดเล็กเท่านั้น หากวงจรที่ทดสอบมีความซับซ้อนมาก วิธีการนี้จะยุ่งยากและเสียเวลามาก เพราะต้องกำหนดค่าสัญญาณต่างๆเองทุกๆ ครั้งที่ต้องการ ได้พยายามมีการแก้ปัญหานี้ด้วยการทำให้โปรแกรมทดสอบการทำงานสามารถบันทึกค่าที่ใช้ในการทดสอบในรูป Test Pattern File เอาไว้ได้ แต่เนื่องจากไฟล์ดังกล่าวมักจะขึ้นอยู่กับโปรแกรมจำลองการทำงานแต่ละโปรแกรมเท่านั้นจึงไม่สามารถนำมาใช้งานร่วมกันกับโปรแกรมอื่นได้ อีกทั้งโปรแกรมที่มีความสามารถนี้มักเป็น โปรแกรมที่มีราคาแพงอีกด้วย

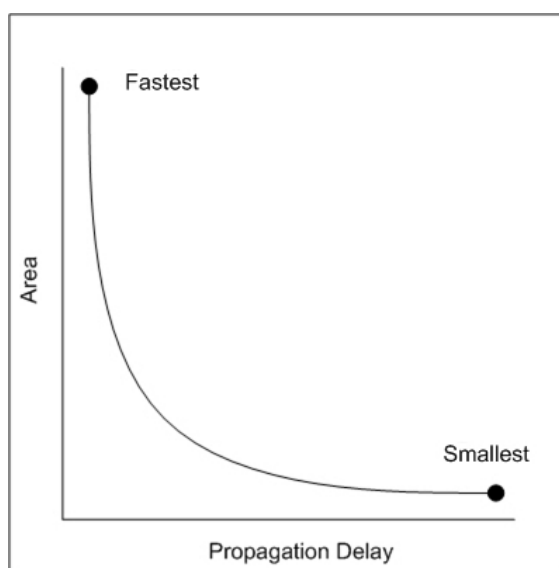
3.2.3.2 เขียนโปรแกรมภาษาวีเอชดีแอลเพื่อทำการทดสอบ

เป็นการเขียนโปรแกรมด้วยภาษาวีเอชดีแอลเช่นกัน แต่โปรแกรมที่เขียนขึ้นใช้ในการทำการทดสอบโดยเฉพาะ ซึ่งทำได้โดยการเขียนเป็นเอนติตี้ขึ้นมาอีกหนึ่งตัว เพื่อทำการเรียกใช้คอมโพเนนท์ตัวที่ต้องการทดสอบ จากนั้นจะทำการสร้างสัญญาณทดสอบ และป้อนให้กับคอมโพเนนท์ที่ต้องการทดสอบเพื่อให้ได้เอาต์พุตออกมาสามารถนำมาเปรียบเทียบกับรูปแบบของค่าเอาต์พุตที่ควรจะเป็นได้ ข้อดีของวิธีการนี้ คือสามารถนำไปใช้ในการจำลองการทำงานกับโปรแกรมจำลองการทำงานตัวใดก็ได้เพราะถูกเขียนขึ้นด้วยภาษาวีเอชดีแอลซึ่งสามารถจำลองการทำงานด้วยโปรแกรมจำลองการทำงานได้ทุกตัว นอกจากนี้ยังช่วยเพิ่มความสะดวกในการจำลองการทำงานเนื่องจากไม่ต้องทำการ Force Input Signal ในการทดสอบเองทุกครั้ง แม้ว่าจะเสียเวลาในการทำการเขียนโปรแกรมทดสอบอยู่บ้าง แต่เมื่อนำไปใช้งานกับวงจรที่มีความซับซ้อนแล้ว ทำให้สามารถจำลองการทำงานได้

รวดเร็วยิ่งขึ้น เพราะเวลาที่เสียไปกับการเขียนโปรแกรมทดสอบนั้นน้อยมาก เมื่อเทียบกับการทำการป้อนค่าทดสอบเองทุกครั้งในการจำลองการทำงาน

3.2.4 Synthesis

ภายหลังจากการทดสอบการทำงานด้วยการจำลองการทำงานแล้ว ว่าโปรแกรมสามารถทำงานได้ตามที่ต้องการก็จะนำโปรแกรมภาษาวีเอชดีแอลเข้าสู่กระบวนการสังเคราะห์วงจรเพื่อทำการสร้างวงจรในระดับเกต โดยวงจรที่ได้จะอยู่ในลักษณะเป็นเน็ตลิสต์ต้องเกิดต่างๆ เชื่อมต่อกัน การสังเคราะห์วงจรสามารถทำได้ใน 2 แบบ คือ แบบที่ไม่ขึ้นกับเทคโนโลยี (Technology-Independent) และแบบที่ขึ้นกับเทคโนโลยี (Technology-Dependent) โดยแบบแรกนั้นวงจรที่ได้สามารถนำไปทำการบันทึกลงอุปกรณ์ที่ใช้สร้างวงจรด้วยเทคโนโลยีแบบใดก็ได้ เช่น ซีพียูแอลดีหรือเอฟพีจีเอ แต่หากทำการสังเคราะห์ตามแบบที่สองนั้นเป็นการกำหนดถึงเทคโนโลยีที่จะใช้ในการออกแบบ จึงต้องทำการกำหนดค่าต่างๆ ให้กับโปรแกรมสังเคราะห์ด้วย เช่น เทคโนโลยีที่ใช้ ชื่อผู้ผลิต หมายเลขรุ่น เพื่อให้โปรแกรมสามารถนำเอาวงจรดังกล่าวมาปรับแต่ง (Optimization) ให้เหมาะสมได้ โดยสามารถปรับแต่งได้ในด้านของขนาดของวงจร (Area Optimization) และความเร็วในการทำงาน (Speed Optimization) ซึ่งผลของการกำหนดค่าในด้านหนึ่งจะมีผลต่ออีกด้านหนึ่งด้วย คือหากกำหนดให้วงจรมีขนาดเล็กที่สุดแล้ว การทำงานของวงจรก็จะช้าที่สุด ในขณะที่ทำการกำหนดให้วงจรมีความเร็วมากที่สุดแล้ว วงจรที่ได้ก็จะมีขนาดใหญ่ที่สุดดังรูปที่ 3.7



รูปที่ 3.7 ความสัมพันธ์ของขนาดและความเร็วของวงจรจากการสังเคราะห์

3.2.5 Place and Route

เป็นการจัดวางและกำหนดการเชื่อมต่อสัญญาณ โดยการแบ่งวงจรที่ได้จากการสังเคราะห์ออกเป็นส่วนย่อยๆ เพื่อให้สามารถบันทึกลงในบล็อกของอุปกรณ์เอฟพีจีเอหรือซีพียูแอลดีที่เลือกใช้ได้ และทำการจัดวาง

ตำแหน่งลงบนอุปกรณ์ให้เหมาะสมที่สุดแล้วทำการเชื่อมต่อสายสัญญาณต่างๆ ภายในวงจรเข้าด้วยกันโดยอาศัยโปรแกรมเข้าช่วย โปรแกรมจะแสดงแผนผังโครงสร้างของอุปกรณ์ที่ใช้ และวงจรที่ออกแบบได้ ทั้งในส่วนของการจัดวางในตำแหน่งต่างๆ ของอุปกรณ์และเส้นทางการเชื่อมต่อสายสัญญาณภายในอุปกรณ์ซึ่งจะเกิดขึ้นจริงเมื่อทำการบันทึกลงไปในตัวอุปกรณ์

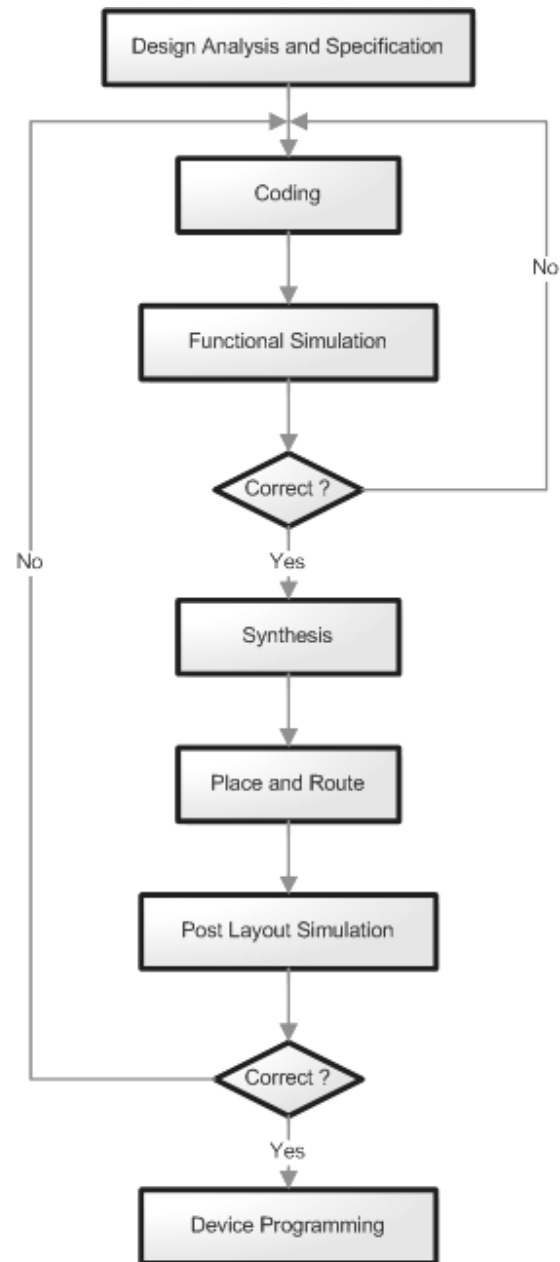
3.2.6 Post Layout Simulation

เมื่อผ่านกระบวนการจัดวางและกำหนดการเชื่อมต่อสัญญาณแล้ว จะได้ชุดข้อมูลซึ่งสามารถนำไปทำการบันทึกลงบนอุปกรณ์ได้ทันที แต่เนื่องจากการทำงานของอุปกรณ์อิเล็กทรอนิกส์นั้นมีค่าของความหน่วงเวลา (Propagation Delay) เกิดขึ้นภายในเสมอ แม้ว่าค่าที่ได้จะอยู่ในช่วงเวลาน้อยมากเพียงระดับนาโนวินาที แต่เมื่อนับรวมทั้งวงจรซึ่งมีขนาดใหญ่มากระดับ 10,000 เกตขึ้นไปแล้ว ค่าหน่วงเวลาดังกล่าวจะมีค่ามากจนอาจทำให้การทำงานของวงจรผิดพลาดได้ หรือไม่สามารทำงานที่ความถี่สูงมากตามที่คาดไว้ได้ จึงต้องทำการทดสอบการทำงานของวงจรดังกล่าวก่อนหลังจากขั้นตอนจัดวางและกำหนดการเชื่อมต่อสัญญาณ หากไม่เป็นไปตามความต้องการแล้วอาจจะเป็นเพราะการเขียนโปรแกรมไม่ดีพอ ซึ่งต้องกลับไปทำการแก้ไขและทำตามขั้นตอนต่างๆ ตั้งแต่การจำลองการทำงาน การสังเคราะห์วงจร และการจัดวางและเชื่อมต่อสัญญาณใหม่ทั้งหมดอีกครั้ง เมื่อผ่านขั้นตอนการทดสอบนี้แล้วแทบจะมั่นใจได้เลยว่าวงจรที่ได้ออกแบบนั้นสามารถนำไปบันทึกลงบนอุปกรณ์ และมีการทำงานที่ถูกต้องตามที่ต้องการ

3.2.7 Device Programming

ขั้นตอนสุดท้ายคือการนำเอาวงจรที่ได้ทำการออกแบบบันทึกลงบนอุปกรณ์ที่เป็นฮาร์ดแวร์จริงๆ ซึ่งอาจอยู่ในรูปของเอฟพีจีเอหรือซีพีแอลดี โดยการนำไฟล์ข้อมูลที่เป็นบิตสตรีมสำหรับอุปกรณ์นั้นๆ มาทำการบันทึกโดยอาศัยเครื่องโปรแกรมโดยเฉพาะหรือสายดาวน์โหลดข้อมูลสำหรับอุปกรณ์นั้นๆ หากอุปกรณ์นั้นใช้เทคโนโลยีแบบ Static RAM ซึ่งข้อมูลจะหายไปเมื่อหยุดจ่ายไฟเลี้ยง จะต้องทำการโหลดข้อมูลลงไปยังอุปกรณ์นั้นใหม่ทุกครั้ง หรืออาจจะบันทึกไว้ใน EEPROM หรือ Serial PROM เพื่อให้ข้อมูลถูกโหลดลงไปเมื่อจ่ายไฟ หากเป็นอุปกรณ์ที่ออกแบบมาแบบ Flash แล้ว ข้อมูลที่ทำการบันทึกก็จะยังคงอยู่แม้ว่าจะตัดไฟเลี้ยงวงจรออกแล้วก็ตาม

เมื่อทำตามขั้นตอนทั้งหมดก็จะได้วงจรที่ออกแบบอยู่ในรูปของฮาร์ดแวร์จริง และมักจะนำไปทดสอบการทำงานอีกครั้ง ซึ่งผลการทำงานของวงจรในเมื่อถึงขั้นตอนนี้แล้วมีความผิดพลาดน้อยมาก การเขียนโปรแกรมด้วยภาษาวีเอชดีแอลจึงเหมาะสมและเป็นที่นิยมในการนำมาพัฒนาออกแบบเป็นอย่างมาก ขั้นตอนในการออกแบบระบบด้วยภาษาวีเอชดีแอลสามารถแสดงเป็น Flow Chart ได้ดังรูปที่ 3.8



รูปที่ 3.8 Flow Chart แสดงขั้นตอนในการออกแบบระบบด้วยภาษาวีเอชดีแอล

3.3 การเขียนโปรแกรมภาษาวีเอชดีแอล

3.3.1 การเขียนภาษาวีเอชดีแอลเบื้องต้น

ภาษาวีเอชดีแอลเป็นภาษาในลักษณะ Case Insensitive คือไม่มีความแตกต่างกันในการเขียนตัวอักษรพิมพ์เล็กหรือพิมพ์ใหญ่ ซึ่งการเขียนแบบใดก็มีความหมายเหมือนกัน ในการเขียนโปรแกรมแต่ละบรรทัดจะต้องจบคำสั่งด้วยเครื่องหมายอัฒภาค (;) และการกำหนดค่าให้กับสัญญาณต่างๆ จะใช้เครื่องหมายน้อยกว่าหรือเท่ากับ ($\leq$) หรือเรียกว่า Signal Assignment โดยค่าทางขวามือจะเป็นค่าอินพุตที่ส่งไปยังค่าเอาต์พุตทางซ้ายมือ ในการประกาศสัญญาณต่างๆ ที่เป็นชนิดเดียวกันสามารถประกาศได้โดยใช้เครื่องหมายจุลภาค (,) คั่นระหว่างสัญญาณแต่ละตัว ในการเขียนคำอธิบายโปรแกรมจะต้องใช้เครื่องหมาย (--) ไว้หน้าบรรทัดในการป้องกันไม่ให้คอมไพเลอร์ทำการอ่านคำสั่งบรรทัดนั้นๆ ส่วนที่สำคัญของภาษาวีเอชดีแอลคือการทำงานแบบขนานของโปรแกรม นั่นคือ การเขียนโค้ดแต่ละบรรทัดจะไม่มีผลในเชิงของตำแหน่งของบรรทัด หมายความว่า คำสั่งต่างๆ นั้นจะทำงานไปพร้อมๆ กันได้ซึ่งสามารถนำไปบรรยายการทำงานของฮาร์ดแวร์จริงได้

3.3.2 องค์ประกอบของภาษาวีเอชดีแอล

ในการเขียนโปรแกรมภาษาวีเอชดีแอลเพื่อกำหนดแบบวงจรดิจิทัลทั้งแบบพื้นฐานหรือวงจรที่มีความซับซ้อน ทุกรวงจรจะต้องเขียนให้อยู่ในรูปของวีเอชดีแอลคอมโพเนนท์ โดยมีหน่วยการออกแบบต่างๆ ดังนี้

3.3.2.1 Entity

ใช้สำหรับการติดต่อระหว่างอุปกรณ์ภายนอกกับวงจรที่เขียนขึ้น รวมทั้งการส่งค่าพารามิเตอร์บางอย่างระหว่างอุปกรณ์ภายนอกกับวงจรด้วย ตัวอย่างของ entity ของ half_adder มีรูปแบบการเขียนดังนี้

```
entity half_adder is
    port (
        a      : in    bit;
        b      : in    bit;
        sum     : out   bit;
        carry   : out   bit;
    );
end half_adder;
```

โดยประเภทของพอร์ตที่สามารถกำหนดได้มี 4 ประเภทคือ in สำหรับรับสัญญาณจากภายนอกมาใช้เป็นอินพุต out สำหรับส่งสัญญาณที่พุดออกไปภายนอก inout สำหรับใช้เป็นตัวอินพุตและเอาต์พุต และ buffer สำหรับเป็นเอาต์พุตชนิดหนึ่งซึ่งสามารถอ่านค่ากลับมาใช้ในการทำงานของวงจรได้ด้วย

3.3.2.2 Architecture

เป็นส่วนที่ใช้ในการเขียนบรรยายพฤติกรรมการทำงานของวงจรที่ต้องการออกแบบโดยมีความสัมพันธ์กับสิ่งที่กำหนดไว้ใน entity ตัวอย่างของ architecture ของ half_adder มีรูปแบบการเขียนดังนี้

```
architecture behavioral of half_adder is
begin
    process(a, b)
        variable x, y : bit;
    begin
        x := a;
        y := b;
        sum <= a xor ;
        carry <= a and b;
    end process
end behavioral;
```

โดยสามารถเขียนโมเดลในการออกแบบได้หลายรูปแบบ เช่น behavioral style (algorithm descriptions), data flow style (RTL descriptions) หรือ structural style (Netlist descriptions)

3.3.2.3 Package and Configuration

สำหรับ package นั้นในการเก็บข้อมูลต่างๆ ตลอดจนโปรแกรมย่อยที่ใช้ในการเขียนรูปแบบการบรรยายวงจรดิจิทัล สามารถถูกเรียกใช้โดย entity หรือ architecture นอกจากนี้ยังสามารถนำคอมโพเนนท์มาตรฐานที่มีอยู่ใน package มาใช้งานได้ด้วย การใช้งาน package นั้นสามารถทำได้โดยการใช้คำสั่ง use เช่น การเรียกใช้ไลบรารีของ ieee และเลือกใช้ package ของ std_logic_1164 สามารถทำได้ดังนี้

```
library ieee;
use ieee.std_logic_1164.all;
```

ส่วนของ configuration นั้นใช้สำหรับการกำหนดค่าต่างๆ เช่นในการกำหนดให้โปรแกรมจำลองการทำงานเลือกใช้ architecture ตัวใดตัวหนึ่งในหลายๆตัวที่อยู่ใน entity เพื่อให้ทำการจำลองการทำงานได้อย่างถูกต้อง ตัวอย่าง เช่น การกำหนด configuration ของ full_adder สามารถทำได้ดังนี้

```

configuration config_full_adder of full_adder is
    for structural
        for u1 : half_adder
            use entity
                work.half_adder(rtl);
        end for;
        for u2 : half_adder
            use entity
                work.half_adder(rtl);
        end for;
        for u1 : or_gate
            use entity
                work.or_gate(rtl);
        end for;
    end for;
end config_full_adder;

```

ข้อสำคัญที่ควรระวังคือการเขียน configuration นั้นใช้ได้เพียงเพื่อการจำลองการทำงานเท่านั้น หากต้องการเขียนวงจรแล้วนำไปทำการสังเคราะห์จะต้องทำการกำหนดค่าของ architecture เพียงตัวเดียวสำหรับ entity นั้นๆ หรือถ้าจะให้เหมาะสมควรเขียนให้แต่ละ entity มีเพียง 1 architecture เท่านั้น

3.3.3 การประกาศคอมโพเนนต์และการเรียกใช้งาน

ในการออกแบบเราสามารถทำการเขียนแต่ละคอมโพเนนต์ไว้ในแต่ละไฟล์ หรือทำการเขียนไว้เป็นไลบรารีแล้วทำการเรียกใช้คอมโพเนนต์นั้นๆ บน entity ที่อยู่ในระดับที่สูงขึ้นไปของการออกแบบได้ โดยการประกาศคอมโพเนนต์จะต้องทำการประกาศไว้ในส่วนของ architecture ให้อยู่เหนือคำสั่ง begin จากนั้นทำการเรียกใช้คอมโพเนนต์ต่างๆ โดยกำหนดการเชื่อมต่อไว้ในส่วนถัดมาหลังจากคำสั่ง begin ตัวอย่างเช่น entity full_adder ทำการประกาศและเรียกใช้การทำงานของคอมโพเนนต์ half_adder และ or_gate สามารถทำได้ดังนี้


```

entity full_adder is
    port (    ain, bin ,cin           : in    bit;
            sum_out, carry_out       : out   bit);
end full_adder;

...

architecture structural of full_adder is
    component half_adder
        port (
            a      : in    bit;
            b      : in    bit;
            sum     : out   bit;
            carry   : out   bit
        );
    end component;

    component or_gate
    ...
    end component;

    signal  n_sum, n_carry1, n_carry2 :bit;
begin
    u1 : half_adder
        port map(
            a => ain;
            b => bin
            sum => n_sum;
            carry => carry1);

    u2 : half_adder
    ...

    u3 : or_gate
    ...

end structural;

```